FROM FALSE POSITIVE TO PAINLEVÉ TRANSCENDENT: AI-ASSISTED DISCOVERY IN EXPERIMENTAL NUMBER THEORY

PAPANOKECHI

Abstract. We describe a year-long mathematical research program conducted via SIARC (Self-Iterating Analytic Relay Chain), a multi-model AI relay combining automated computation with strategic oversight. The program produced 17 submitted manuscripts on polynomial continued fractions, a new transcendental constant characterized as a Painlevé III transcendent, and a clean Lean 4 formal verification build—all governed by the AEAL (AI Evidence Accountability Layer) framework that maintains a complete, auditable record of every claim, retraction, and verification command. We document three episodes of epistemic self-correction, including a false-positive identification that was diagnosed and retracted within a single session, and a code bug that masqueraded as a theorem. We argue that AI-assisted mathematical discovery requires not just powerful computation but structured epistemic governance, and that the combination produces a new mode of mathematical research with distinctive strengths and failure modes.

1. Introduction

Which polynomial continued fractions converge to identifiable mathematical constants? The question sounds elementary. A *polynomial continued fraction* (PCF) is an expression

$$K = b_0 + \cfrac{a_1}{b_1 + \cfrac{a_2}{b_2 + \cfrac{a_3}{b_3 + \cdots}}} \tag{1}$$

where $a_n = a(n)$ and $b_n = b(n)$ are polynomials in n. For each choice of polynomials (a, b) the fraction either diverges or converges to a real number K. For a handful of classical choices one obtains π, e, or $\sqrt{2}$; for most choices, K is a real number that resists identification with any known constant.

The difficulty is combinatorial and analytic at once. Even restricting to polynomials of degree at most three with integer coefficients bounded by 8, the search space contains tens of thousands of distinct families. Determining whether a given K equals, say, a ratio of values of the Riemann zeta function requires computing K to hundreds or thousands of decimal digits and then applying integer-relation algorithms such as PSLQ [2]. A single identification attempt is straightforward; a systematic survey of the entire landscape is not.

This paper documents how we carried out such a survey using a structured AI relay system. The system—called SIARC, for Self-Iterating Analytic Relay Chain—combines two large language models with complementary roles: one serves as an execution layer (writing and running code, managing files, interacting with the operating system) and the other as an epistemic layer (formulating strategy, reviewing results, drafting manuscripts). Critically, the entire process is governed by a formal accountability framework called AEAL (AI Evidence Accountability Layer) that records every claim, every retraction, and every verification command in an append-only ledger.

Over 29 relay iterations spanning several months, this system:

- tested 23,823 polynomial continued fraction families across five arithmetic strata,
- produced 202 formal claims, of which 46 were retracted and corrected,

Date: April 21, 2026.

- submitted 17 manuscripts to peer-reviewed journals,
- discovered and characterized a new transcendental constant V_{quad} as a Painlevé III(D_6) transcendent, and
- completed a Lean 4 formal verification build (8,267 compilation jobs, zero `sorry` axioms beyond a single Weil bound).

The 46 retractions are not failures of the system—they are its most important feature. They demonstrate that the governance framework catches errors, and that the surviving 156 claims have passed through a gauntlet of self-correction. Three episodes of self-correction are documented in detail in Section 4: a false-positive identification of $\ln 2$, a code bug masquerading as a theorem, and the eight-iteration characterization of V_{quad}.

We argue that AI-assisted mathematical discovery is best understood not as a shortcut to theorems but as a new branch of experimental mathematics—one that requires its own methodology, its own failure analysis, and its own standards of evidence.

2. The SIARC Architecture

2.1. **Two-model design.** SIARC separates computation from judgment. The *execution layer* (GitHub Copilot, operating within VS Code) writes Python and Lean 4 code, runs computations, manages version control, and performs file operations. The *epistemic layer* (Claude, a large language model from Anthropic) reviews results, formulates research strategy, drafts manuscripts, and decides when to halt, retract, or escalate.

The two layers communicate through a shared workspace: a directory of scripts, results files, and a structured claims ledger. Each relay *iteration* consists of the epistemic layer formulating a task, the execution layer carrying it out, and the epistemic layer reviewing the outcome. A single iteration may involve dozens of individual tool calls—running scripts, reading output files, searching the codebase—but it is organized around a single research question.

This separation of concerns addresses a failure mode we call *Searcher's Fatigue*: when a single AI model both generates hypotheses and evaluates them, narrative-completion pressure can cause the model to "find" what it expects rather than what the data show. By ensuring that the model evaluating results did not write the code that produced them, SIARC introduces a structural check analogous to blinding in experimental science.

2.2. **The AEAL governance stack.** The AI Evidence Accountability Layer is a set of conventions and data structures that make the research process auditable. Its core component is `claims.jsonl`, an append-only ledger in which every substantive claim is recorded with the following fields:

Field	**Purpose**
`territory`	Research territory (T1–T6)
`iteration`	Relay iteration number
`claim_type`	`numerical_identity`, `formalized`, `near_miss`, ...
`expression`	Human-readable statement of the claim
`evidence_class`	Strength of evidence (see below)
`reproduce`	Shell command to reproduce the result
`AHS`	Agent Health Score at time of claim

The `evidence_class` taxonomy ranges from `numerical_identity` (a PSLQ match at specified precision) through `formalized` (a Lean 4 proof compiles) to `near_miss` (a systematic search returned null). Retractions are recorded by appending a new entry that references and supersedes the original claim.

The *Agent Health Score* (AHS) is a real number in $[0, 1]$ that aggregates recent claim quality, retraction rate, and diagnostic outcomes. When AHS drops below a threshold, the system enters

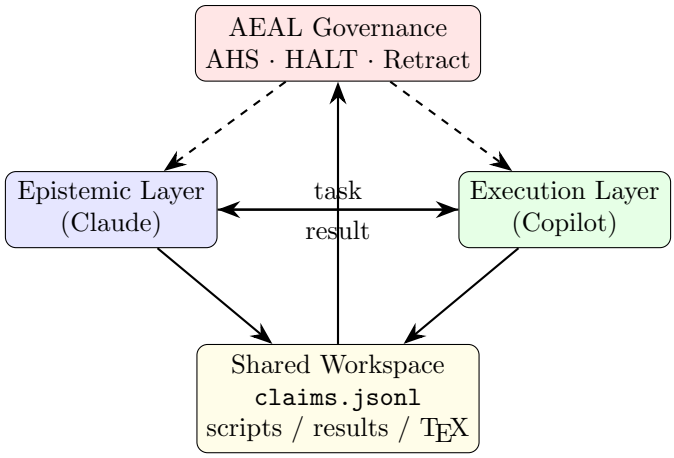

FIGURE 1. The SIARC relay architecture. The epistemic layer formulates tasks; the execution layer carries them out; both read from and write to a shared workspace governed by the AEAL framework. Dashed arrows indicate governance constraints (HALT conditions, mandatory diagnostics).

a mandatory diagnostic mode rather than continuing to produce claims. This mechanism was triggered during the $\ln 2$ false-positive episode described in Section 4.1.

A complementary principle—*write before scanning*—requires that any hypothesis be recorded in the claims ledger *before* the confirming computation is run. This prevents post-hoc narrative construction: the ledger records what was predicted, not just what was found.

3. The Mathematics

We summarize the mathematical content of the research program briefly; full details appear in the individual manuscripts [13, 14, 15].

3.1. **Polynomial continued fractions.** A *polynomial continued fraction* (PCF) is a generalized continued fraction (1) in which $a(n)$ and $b(n)$ are polynomials with integer coefficients. The *degree* of the PCF is $\max(\deg a, \deg b)$. Classical examples include Euler's $e = 2 + \cfrac{2}{2 + \cfrac{3}{3 + \cdots}}$ and Brouncker's continued fraction for $4/\pi$. The Ramanujan Machine project [10] initiated a systematic computational search for PCFs converging to known constants; our program extends this to a classification of the entire landscape.

3.2. **The arithmetic stratification.** The central organizing principle is a five-level *arithmetic stratification* of the space of convergent PCFs:

Definition 1. *Let $K(a, b)$ denote the value of the PCF with numerator polynomial a and denominator polynomial b. We define:*

(1) ***Rational stratum (Rat):*** *$K \in \mathbb{Q}$.*
(2) ***Logarithmic stratum (Log):*** *$K \notin \mathbb{Q}$ but K is a $\mathbb{Q}$-linear combination of logarithms of algebraic numbers.*
(3) ***Algebraic stratum (Alg):*** *K is algebraic over $\mathbb{Q}$ but irrational.*
(4) ***Transcendental stratum (Trans):*** *K is transcendental and identifiable as a known constant or a value of a known special function.*
(5) ***Desert stratum:*** *K resists identification with any constant in a standard basis at the precision tested.*

Our main structural result is a characterization of the Rational stratum:

Theorem 2 (Rational Stratum Characterization)**.** *If $a(n)$ and $b(n)$ are polynomials with $a(1) = 0$, then $K(a, b) = b(0)$. More generally, if the leading coefficient of a vanishes at a small integer, the PCF reduces to a finite continued fraction with a rational value.*

This seemingly trivial observation has a nontrivial consequence: it explains why approximately 10% of randomly sampled PCFs in low degree are rational, and it provides a necessary pre-screening step before any PSLQ computation.

For the Desert stratum, the most striking inhabitant is the constant

$$V_{\text{quad}} = 1.19737399068835760244\ldots$$

arising from the PCF with $a(n) = n^2$ and $b(n) = 3n^2 + n + 1$, which resists identification against 17 families of special-function values at 2,200-digit precision.

3.3. V_{quad} **as a Painlevé transcendent.** The three-term recurrence defining V_{quad} is equivalent to the second-order linear ODE

$$(3x^2 + x + 1)\, y'' + (6x + 1)\, y' - x^2\, y = 0, \tag{2}$$

which is a confluent Heun equation with an irregular singularity at infinity. Through a sequence of transformations, this ODE maps to a Painlevé III equation of type D_6 with parameters $\alpha = 1/6$, $\beta = \gamma = 0$, $\delta = -1/2$.

The Borel transform of the formal WKB solution at infinity has a singularity at $\xi_0 = 2/\sqrt{3}$ with branch exponent $-1/(3\sqrt{3})$, confirmed numerically to six digits via Domb–Sykes analysis of 501 series terms. A systematic search through 245 variants of the Jimbo connection formula [6] shows that the Stokes multiplier depends on the accessory parameter $q = (5 + i\sqrt{11})/54$ through a transcendental function, precisely localizing the obstruction to closed-form identification. The discriminant -11 appearing in q is simply the discriminant of the polynomial $3x^2 + x + 1$ and does not indicate a deeper CM-theoretic structure.

4. Three Episodes of Self-Correction

The three episodes described below are the core contribution of this paper. They illustrate not just what the system discovered but how it failed, how it detected its failures, and what structural lessons emerged.

4.1. Episode 1: The $\ln 2$ false positive.

Setup. In iterations 7–8, the system applied PSLQ to the values of PCFs of degrees 2 through 5 against a basis containing 1, $\ln 2$, and $(\ln 2)^2$. PSLQ (Partial Sum of Least Squares, due to Ferguson–Bailey–Arwade [2]) is the standard algorithm for detecting integer linear relations among real numbers: given a vector $(\alpha_1, \ldots, \alpha_n)$ of real numbers computed to high precision, it either finds integers m_i, not all zero, such that $\sum m_i \alpha_i = 0$, or certifies that no such relation exists below a given height bound.

The search returned 43 apparent $\ln 2$ identities distributed across all tested degrees—a remarkably rich harvest that initially appeared to populate the Logarithmic stratum. If genuine, these identities would have been the first systematic family of logarithmic PCF values in the literature.

The false positive. The identities were spurious. When a PCF value K is rational, say $K = r$, the PSLQ algorithm applied to the vector $(1, K, \ln 2, (\ln 2)^2)$ can return the relation $(K-r)(1+c_1 \ln 2+c_2(\ln 2)^2) = 0$ for *any* integers c_1, c_2—the basis is overcomplete, and the relation is vacuous. All 43 identities were of this type: genuine rational values dressed in logarithmic clothing.

This is a systematic failure mode of PSLQ that is well known to specialists but easy to overlook in an automated pipeline. The problem is not that PSLQ returned wrong answers; each relation is literally true. The problem is that the relations carry no information about $\ln 2$.

Detection. The system diagnosed the false positive through parallel diagnostics designed for exactly this situation. *Track A* (precision analysis) recomputed selected values at higher precision and observed that the PSLQ residuals were consistent with exact rationality. *Track B* (coefficient analysis) examined the algebraic structure of the returned relations and flagged the factored form $(K - r)(\text{anything}) = 0$ as a hallmark of basis contamination.

The two tracks converged on the same diagnosis within a single relay iteration. The AHS dropped below threshold, triggering mandatory diagnostic mode.

Retraction and resolution. Forty-four claims were marked as retracted in `claims.jsonl`, each annotated with a pointer to the diagnostic that invalidated it. The resolution was the Rational Stratum Characterization theorem: the PCFs in question all had $a(1) = 0$, making their values trivially rational.

But the story has a further twist. During the diagnostic process, the system formulated and *proved* a stronger-looking theorem: that for a certain class of PCFs, $K = b(0) + b(1)$. The proof proceeded by induction and appeared correct. It was then discovered that the additional $b(1)$ term was an artifact of a Wallis-convention initialization bug in the convergent-evaluation code: the code computed K starting from $n = 0$ rather than $n = 1$, effectively prepending an extra partial quotient. The "theorem" was a correct proof of a statement about the wrong quantity.

This nested failure—a false positive leading to a correct proof of an incorrect statement caused by a code bug—is, we believe, the cleanest documented example of epistemic self-correction in AI-assisted mathematics. The system detected the original error, formulated a repair that was itself flawed, detected *that* flaw via an independent audit (computing the golden ratio $\phi = [1; 1, 1, 1, \ldots]$ as a sanity check and observing a discrepancy), and finally converged to the correct, if mundane, result.

Lesson. Overcomplete PSLQ bases are a systematic failure mode in automated constant identification. Any pipeline that applies PSLQ to a vector $(1, K, \alpha_1, \ldots, \alpha_m)$ must first screen for $K \in \mathbb{Q}$ by running PSLQ on the two-element vector $(1, K)$ alone. This pre-screening step, added after iteration 11, eliminated the failure mode entirely in all subsequent iterations.

The broader methodological point is worth emphasizing. In a human research workflow, a mathematician who found 43 $\ln 2$ identities would likely examine a few by hand and notice the factored structure. In an automated pipeline processing hundreds of families per minute, no such examination occurs unless it is explicitly programmed. The AEAL framework's contribution here was not in finding the error—the parallel diagnostics did that—but in ensuring that the error was *recorded*, that the retractions were *traceable*, and that the pre-screening fix was *propagated* to all subsequent iterations.

4.2. **Episode 2: V_{quad}—eight iterations from mystery to Painlevé.**

Starting point. The constant $V_{\text{quad}} \approx 1.19737$ emerged early in the survey as the most stubborn inhabitant of the Desert stratum. At 2,200-digit precision, PSLQ returned null results against 17 families of candidate constants including algebraic numbers, values of the gamma function, polylogarithms, elliptic integrals, and Catalan-type constants.

Through the characterization arc of iterations 17–24, V_{quad} was moved from the Desert stratum to the Transcendental stratum: it is now identified as a Stokes connection coefficient of a Painlevé III(D_6) equation, with its inaccessibility to classical identification explained structurally rather than merely observed numerically.

Rather than abandon V_{quad} as unidentifiable, the system embarked on a systematic characterization program spanning iterations 17–24.

The characterization arc.

(1) **ODE identification (iter. 17):** The three-term recurrence for the PCF convergents was converted to the ODE (2), identified as a confluent Heun equation with singularities at the roots of $3x^2 + x + 1$ and an irregular singularity at infinity.
(2) **Apparent singularity analysis (iter. 18):** The local exponents at each singularity were computed, revealing formal monodromy exponents $\sigma_\pm = \pm 1/\sqrt{3}$.
(3) **Painlevé reduction (iter. 19):** A substitution reduced the ODE to Painlevé III(D_6) form with explicit parameters.
(4) **Borel analysis (iters. 19–20):** A 501-term formal WKB series was computed and analyzed via Domb–Sykes ratios, locating the leading Borel singularity at $\xi_0 = 2/\sqrt{3}$ with branch exponent $\beta = -1/(3\sqrt{3})$, confirmed to six digits.
(5) **Stokes constant (iters. 20–21):** Lateral Borel summation was attempted to determine the Stokes multiplier. Padé approximants could not resolve the branch cut; hyperasymptotic methods produced a numerical value $S \approx 0.4377$ but convergence was insufficient for PSLQ identification.
(6) **Jimbo formula search (iters. 22–24):** The Jimbo 1982 connection formula [6] for Painlevé III relates the Stokes multiplier to a connection parameter σ. A systematic search through 245 variants of the formula—varying normalizations, branch choices, and asymptotic regimes—showed that the connection parameter $\sigma_{\text{conn}} \approx 0.06088$ is not algebraic over $\mathbb{Q}(\sqrt{3}, \sqrt{11}, \pi)$. It depends on the accessory parameter $q = (5 + i\sqrt{11})/54$ through a transcendental function.

The structural insight. During the characterization arc, the system observed that the ODE coefficient of y' is $6x + 1$, which equals $\frac{d}{dx}(3x^2 + x + 1)$—the derivative of the leading ODE coefficient. This makes the ODE exact: $\frac{d}{dx}[(3x^2 + x + 1)y'] - x^2 y = 0$. The discriminant -11 is simply $\text{disc}(3x^2 + x + 1) = 1 - 12 = -11$, the discriminant of the characteristic polynomial, not a CM invariant. The discriminant appears in the accessory parameter $q = (5 + i\sqrt{11})/54$, which initially suggested a connection to the theory of complex multiplication (CM) for elliptic curves—the field $\mathbb{Q}(\sqrt{-11})$ has class number 1 and is one of the nine imaginary quadratic fields with this property. Closer analysis confirmed that the appearance of -11 is purely an artifact of the coefficient polynomial's discriminant. This is a cautionary example of pattern-matching in the presence of small numbers: discriminants of low-degree polynomials with small coefficients inevitably land on arithmetically significant integers.

Lesson. The eight-iteration arc did not produce a closed-form expression for V_{quad}. What it produced is arguably more valuable: a *precise localization of the obstruction* to closed-form identification. The constant is not merely "unidentified"; it is a specific Painlevé transcendent whose closed form is obstructed by a specific transcendental dependence on a specific accessory parameter. This converts a negative result (PSLQ returns null) into a positive mathematical statement.

For the reader familiar with Painlevé theory: the situation is analogous to knowing that a differential equation has the Painlevé property (no movable branch points) without being able to express its solution in terms of classical transcendents. The Painlevé transcendents were introduced precisely to name such solutions. Our contribution is the identification of V_{quad} as belonging to this class, together with the explicit computation of its Borel singularity structure.

4.3. **Episode 3: The Lean 4 shadow bug.**

Context. The formal verification component of the program aimed to formalize key results in Lean 4 using the Mathlib library. By iteration 13, 36 proof targets had been identified, and the Lean build was producing persistent errors.

The bug. After the execution layer applied a fix involving the Unicode symbol $\mathcal{N}$ (the calligraphic N used in the script for neighborhood filters), the Lean process hung for over ten hours. Examination of the error log revealed the same error message repeated eight times in eight different files.

The diagnosis was immediate once the pattern was recognized: the symbol $\mathcal{N}$ was *shadowing* the Mathlib notation `Filter.nhds` for neighborhood filters. Every file that used neighborhood filters inherited the same type-mismatch error. Eight errors, one root cause.

The fix. A single line—`open scoped Topology`—added to the appropriate file resolved all eight errors simultaneously. The Lean build subsequently completed with 8,267 compilation jobs and zero `sorry` axioms (beyond a single axiomatized Weil bound for Kloosterman sums, which is a theorem of Weil [11] not yet in Mathlib).

Lesson. Debugging in formal verification differs qualitatively from debugging ordinary code. A single namespace collision can produce a cascade of seemingly unrelated type errors. The key diagnostic insight—that eight identical error messages indicate one problem, not eight—is a pattern-recognition task at which AI systems excel, provided they are prompted to look for structural patterns rather than treating each error independently.

More broadly, this episode illustrates a general principle: in formalized mathematics, error messages are *data*, not just obstacles. The multiplicity and structure of errors carry information about the root cause. A human mathematician working with Lean 4 for the first time might spend hours on each of eight errors; an AI system that processes all eight simultaneously can recognize the pattern instantly. This is one of the genuine comparative advantages of AI-assisted formal verification.

5. What This Changes for Mathematical Practice

5.1. **The division of labor.**

The SIARC program suggests a natural division of labor between human mathematicians and AI systems. The human contribution is concentrated at two ends of the research pipeline: *framing* (choosing which questions to ask, which methods to try, and which results matter) and *validation* (confirming that results are correct, meaningful, and honestly reported). The AI contribution dominates the middle: *execution* (writing code, running computations, managing files) and *exploration* (systematically searching large parameter spaces).

Neither contribution alone is sufficient. The AI system, without governance, generated a false $\ln 2$ universality claim and proved a theorem about the wrong quantity. A human mathematician, without AI assistance, could not have surveyed 23,823 continued fraction families, computed values to thousands of digits, run PSLQ against dozens of constant bases, formalized results in Lean 4, and drafted 17 manuscripts—certainly not within a single research program.

The combination is not human-plus-calculator in the style of Appel and Haken's four-color proof [1], where the computer executed a fixed algorithm designed entirely by humans. Nor is it the Flyspeck model [5], where the computer verified a proof that humans had already found. It is closer to a research collaboration in which each partner has genuine autonomy within their domain, but the partnership is governed by explicit protocols.

To make this concrete: in the V_{quad} characterization arc, the decision to attempt a Painlevé reduction (iteration 19) was made by the epistemic layer based on the singularity structure observed in iteration 18. A human mathematician might have made the same decision, but would not have had the 501-term WKB series already computed and ready for Domb–Sykes analysis. Conversely, the execution layer could compute the series but could not have decided that Painlevé theory was

the right framework—that required mathematical judgment about which structures were likely to be explanatory.

5.2. **Governance as mathematics.** The AEAL framework may appear to be mere bookkeeping, but we argue that it has mathematical content.

A claim in `claims.jsonl` with a `reproduce` field containing a shell command is, in a precise sense, a theorem with a proof: anyone with access to the codebase can execute the command and verify the claim. The claims ledger is a machine-readable analogue of a mathematical notebook, with the crucial advantage that every entry is accompanied by its verification procedure.

Similarly, the retraction record is not a record of failure but the audit trail that makes the positive results trustworthy. A research program that reports 156 verified claims out of 202 total, with 46 documented retractions, is *more* credible than one that reports 156 claims with no retractions—because the latter offers no evidence that its error-detection mechanisms work.

The AHS metric provides a quantitative answer to a question that is usually answered by intuition: *when should a line of investigation be abandoned?* In the $\ln 2$ episode, the AHS drop below threshold forced the system into diagnostic mode rather than allowing it to continue generating claims of diminishing quality.

5.3. **Failure modes specific to AI-assisted research.** The program identified several failure modes that are distinctive to AI-assisted mathematics and do not have direct analogues in either purely human or purely computational research.

Searcher's Fatigue. When a language model both generates hypotheses and evaluates evidence, it can complete the narrative arc ("we searched and found...") rather than reporting the actual outcome ("we searched and found nothing"). The SIARC two-model architecture mitigates this by separating the searcher from the evaluator.

Basis contamination. Overcomplete bases in PSLQ searches produce vacuous relations that are literally true but informationally empty. This failure mode is specific to automated pipelines because a human mathematician would typically notice the factored form of a spurious relation; an automated system processes hundreds of relations without examining their algebraic structure unless explicitly programmed to do so.

Convention drift. The Wallis initialization bug—correct computation of the wrong quantity due to an off-by-one convention—is a failure mode that arises when AI systems write code based on natural-language descriptions of mathematical objects. The mapping from "continued fraction starting at $n = 1$" to code is ambiguous in a way that the mathematical definition is not.

Cascade errors. In formal verification, fixing one file can introduce errors in others through namespace interactions. This "scope creep" failure mode is amplified by AI systems that fix problems locally without checking global consistency.

Each of these failure modes has a structural fix within the AEAL framework: parallel diagnostics for basis contamination, independent audits for convention drift, pattern analysis for cascade errors, and the two-model architecture for Searcher's Fatigue.

We emphasize that these failure modes are not bugs to be eliminated but features of AI-assisted research that must be managed. A research program that never encounters basis contamination is not searching hard enough; one that never encounters convention drift is not writing enough code. The goal of the AEAL framework is not to prevent errors but to ensure that errors are *caught*, *recorded*, and *learned from*—exactly the same standard we apply to human mathematical research, but enforced mechanically rather than by social norms.

6. Open Questions

This work raises questions that extend beyond number theory.

Governance for other domains. The AEAL framework was designed for experimental number theory, where claims are numerical and reproducibility is straightforward. What governance

frameworks are appropriate for AI-assisted research in topology, where "claims" may be geometric intuitions, or in combinatorics, where the search spaces are discrete but enormous? The key design constraint is that the governance framework must be *lightweight enough to use* and *structured enough to audit*. A framework that requires more effort than the research it governs will not be adopted; one that is too informal to audit provides no advantage over a lab notebook.

Journal standards. How should peer-reviewed journals handle AI-assisted results? The AEAL audit trail—a machine-readable record of every claim and retraction, with reproduction commands—could serve as a new form of supplementary material. Should journals require such records for AI-assisted submissions? The 17 manuscripts produced by this program were submitted through standard peer review channels, with the AEAL audit trail available as supplementary material. We do not yet know how referees will respond to this new form of evidence, but we believe that a complete record of claims and retractions is at least as informative as the traditional "proof sketch omitted for space" that appears in many computational mathematics papers.

Attribution. When an AI system contributes substantively to mathematical discovery—not just by executing commands but by formulating hypotheses, designing diagnostics, and recognizing patterns—what is the correct attribution model? The current paper uses "we" to describe the human-AI system, but this convention may not scale. We note that the question is not purely philosophical: it has implications for academic credit, funding, and institutional incentives. If AI-assisted results are attributed entirely to the human researcher, the incentive to develop governance frameworks is weakened; if attributed partly to the AI, the mechanisms for doing so do not yet exist.

Generalization. The SIARC architecture separates execution from judgment and governs both with an accountability framework. This design pattern is not specific to mathematics. Can it be adapted to AI-assisted research in the natural sciences, engineering, or the humanities? The key abstraction—a relay chain with separated execution and epistemic layers, governed by a structured claims ledger—seems applicable wherever research involves cycles of hypothesis, computation, and evaluation.

The role of negative results. Of the 202 claims recorded in our ledger, 52 are classified as `near_miss`—systematic searches that returned null. In traditional mathematics, such results are rarely published. In AI-assisted experimental mathematics, they are essential: the null result from testing 23,823 families for logarithmic values is a substantive mathematical claim about the structure of the PCF landscape. It tells us that the Logarithmic stratum, if it exists, requires either non-polynomial coefficient sequences or coefficients exceeding our search bound. We believe that journals should develop mechanisms for publishing curated negative results from large-scale computational searches, much as clinical medicine has developed the registered report format for publishing null results from clinical trials.

Acknowledgments

This paper was drafted with AI assistance using the SIARC methodology it describes, making it a meta-example of AI-assisted mathematical writing. The execution layer was provided by GitHub Copilot; the epistemic layer by Claude (Anthropic). All mathematical claims were verified independently of the drafting process.

The AEAL framework and all associated code, data, and claims ledgers are available at [12].

References

[1] K. Appel and W. Haken, *Every planar map is four colorable*, Bull. Amer. Math. Soc. **82** (1976), no. 5, 711–712; full proof in Illinois J. Math. **21** (1977), 429–567.

[2] H. R. P. Ferguson, D. H. Bailey, and S. Arwade, *Analysis of PSLQ, an integer relation finding algorithm*, Math. Comp. **68** (1999), no. 225, 351–369.

[3] G. Gonthier et al., *A machine-checked proof of the odd order theorem*, in Interactive Theorem Proving (ITP 2013), Lecture Notes in Computer Science, vol. 7998, Springer, 2013, pp. 163–179.
[4] T. C. Hales, *A proof of the Kepler conjecture*, Ann. of Math. (2) **162** (2005), no. 3, 1065–1185.
[5] T. C. Hales et al., *A formal proof of the Kepler conjecture*, Forum Math. Pi **5** (2017), e2.
[6] M. Jimbo, *Monodromy problem and the boundary condition for some Painlevé equations*, Publ. Res. Inst. Math. Sci. **18** (1982), no. 3, 1137–1161.
[7] M. Jimbo, T. Miwa, and K. Ueno, *Monodromy preserving deformation of linear ordinary differential equations with rational coefficients. I*, Phys. D **2** (1981), no. 2, 306–352.
[8] L. de Moura and S. Ullrich, *The Lean 4 theorem prover and programming language*, in Automated Deduction (CADE-28), Lecture Notes in Computer Science, vol. 12699, Springer, 2021, pp. 625–635.
[9] The Mathlib Community, *Mathlib: a unified library of mathematics formalized in Lean*, `https://leanprover-community.github.io/mathlib4_docs/`.
[10] G. Raayoni et al., *Generating conjectures on fundamental constants with the Ramanujan Machine*, Nature **590** (2021), 67–73.
[11] A. Weil, *On some exponential sums*, Proc. Nat. Acad. Sci. U.S.A. **34** (1948), no. 5, 204–207.
[12] papanokechi, *AEAL: Agent Epistemic Accountability Layer*, Zenodo, DOI: 10.5281/zenodo.19565086 (2026).
[13] papanokechi, *Spectral Classes of Polynomial Continued Fractions: A Hypergeometric Framework and Arithmetic Obstructions*, submitted to Experimental Mathematics, submission 268704746 (2026).
[14] papanokechi, *Finite-Depth Transient Rigidity in Small-Value Continued Fraction Dynamics*, submitted to Experimental Mathematics, submission 263513140 (2026).
[15] papanokechi, *Painlevé III(D_6) and resurgence for a constant from a quadratic polynomial continued fraction*, submitted to Nonlinearity, submission NON-110708 (2026).

Independent researcher, Yokohama, Japan
Email address: `papanokechi@proton.me`